

CIRCUITS LOGIQUES COMBINATOIRES



Table des matières

Objectifs	4
Introduction	5
I - Matérialisation des fonctions logiques	6
1. Les types de logiques.....	6
2. Les familles logiques	6
3. Analyse et synthèse de circuits logiques combinatoires	7
II - Additionneurs	9
1. Demi-additionneur	9
2. Additionneur complet	10
3. Addition de deux nombres binaires de n bits	11
III - Soustracteurs	12
1. Demi-soustracteur	12
2. Soustracteur complet	13
IV - Multiplexeurs	14
1. La fonction du multiplexage.....	15
2. Multiplexeur 4 vers 1.....	15
3. Multiplexeur à 8 entrées (Mux 74151)	16
V - Démultiplexeurs	18
1. Démultiplexeur 1 vers 4.....	19
2. Application des multiplexeurs et des démultiplexeurs.....	19
VI - Codeurs	21
1. Principe d'un codeurs 4 voies d'entrées et 2 bits de sortie	21
2. Codeur de priorité 8 vers 3.....	23
3. Codage Décimal-BCD.....	23
VII - Décodeurs	26
1. Décodeur élémentaire 2 vers 4 ou décodeur 2 parmi 4.....	26
2. Décodeur 1 parmi 8	28
3. Décodage BCD-Decimal (1 parmi 10)	29
4. Décodeur BCD 7 segments	30

VIII - Transcodeurs	33
1. Réalisation pratique d'un transcodeur	33
2. Exemple de transcodeurs	34
IX - Comparateurs	36
1. Comparateur élémentaire de deux nombres de 1 bits	36
2. Comparateur de deux nombres de n bits.....	37

Objectifs



À la fin de cette leçon, vous serez capables de comprendre les concepts de base qui fondent la logique des circuits combinatoires.

Plus spécifiquement, vous comprendrez :

- comprendre les additionneurs et les soustracteurs ;
- les multiplexeurs et les démultiplexeurs ;
- les encodeurs, les décodeurs et les transcodeurs
- les comparateurs.

Introduction



La transmission de données nécessite fréquemment des opérations de conversion, de transpostage et d'aiguillage. On utilise pour cela des circuits combinatoires. Pour réaliser un circuit logique combinatoire, le concepteur doit utiliser plusieurs portes logiques élémentaires. Pour faciliter sa tâche, les fabricants fournissent des circuits sous forme intégrés comportant chacun plusieurs portes logiques à des degrés d'intégration différents. Il existe plusieurs dispositifs logiques combinatoires couramment utilisés dans les systèmes numériques. On peut citer les codeurs, les décodeurs, les transcodeurs, les multiplexeurs, les démultiplexeurs, les comparateurs ...

La logique combinatoire concerne l'étude des fonctions dont la valeur de sortie ne dépend que de l'état logique des entrées se traduisant par une modification de la valeur des sorties et non pas non plus de ses états antérieurs : à chaque combinaison des variables d'entrée correspond toujours une seule combinaison des fonctions de sortie.

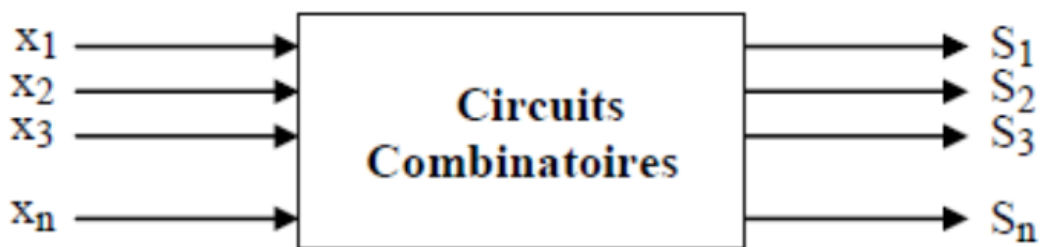


Schéma d'un circuit combinatoire

Matérialisation des fonctions logiques



1. Les types de logiques

À toute grandeurs physiques ayant deux grandeurs possibles, on peut associer une variable booléenne. En électronique, les variables considérées sont essentiellement la tension électrique et le courant électrique. Par exemple, la tension collecteur d'un transistor NPN alimenté sous 5 volts (car de la famille TTL) et fonctionnant en régime de commutation (régime de fonctionnement en logique) peut valoir :

$$\begin{cases} V_C = 0 \text{ V} & \text{si T est saturé} \\ V_C = 5 \text{ V} & \text{si T est bloqué} \end{cases}$$

On peut par convention admettre que la variable booléenne x associée à la tension V_C vaut 1 si $V_C = +5 \text{ V}$ et 0 si $V_C = 0 \text{ V}$.

La valeur 1 est associée à la valeur la plus élevée de V_C : on dit qu'on a défini une **logique positive**.

$$\begin{cases} x = 0 & \text{pour } V_C = 0 \text{ V} \\ x = 1 & \text{pour } V_C = +5 \text{ V} \end{cases}$$

On peut également définir une **logique négative**.

$$\begin{cases} x = 0 & \text{pour } V_C = +5 \text{ V} \\ x = 1 & \text{pour } V_C = 0 \text{ V} \end{cases}$$



Il faut noter que la logique négative est moins utilisée que la logique positive. Pour la suite, nous travaillerons toujours en logique positive.

La logique binaire basée sur l'algèbre de Boole permet de décrire dans un modèle mathématique, les manipulations et traitement des informations binaires et d'analyser les systèmes numériques.

2. Les familles logiques



En électronique numérique, une **famille logique** désigne un ensemble de circuits numériques partageant une technologie et des caractéristiques similaires, notamment en termes de vitesse, de consommation d'énergie, de tension de fonctionnement et de compatibilité.

Toutes les fonctions booléennes peuvent être construites à l'aide trois opérateurs de base AND, OR et NOT. Ce groupe de trois opérateurs forme ce qu'on appelle un système logique complet. La matérialisation des fonctions logiques nécessite donc de pouvoir réaliser des systèmes physiques remplissant ces trois fonctions. Un système logique complet permettant la construction de toute fonction peut cependant être réalisé en utilisant un nombre plus faible de structures de base.

Il existe 4 technologies principales :

- Logique à éléments discrets (diodes et transistors) ;
- Technologie TTL (Transistor-Transistor Logic ;
- Technologie CMOS (Complementary Metal-Oxide-Semiconductor) ;
- Technologie ECL (Emitter-Coupled Logic) ;

Pour plus d'informations sur les familles logiques, cliquez ici¹.

Comparaison des principales familles logiques

Famille	Vitesse	Consommation	Tension d'alimentation	Applications
TTL	Moyenne	Moyenne à élevée	5 V	Circuits généraux
ECL	Très rapide	Très élevée	5 V	Télécommunication superordinateurs
CMOS	Moyenne à élevée	Très faible (statique)	3-15 V	Systèmes modernes (Microprocesseurs)
BiCMOS	Elévée	Moyenne	3-5 V	Pprocesseurs hautes performances
RTL/DTL	Lente	Elevée	5 V	Obsolètes

Choix d'une famille logique

Le choix dépend des critères suivants :

1. **Vitesse** : Pour les applications critiques en termes de temps (ECL, CMOS haute vitesse).
2. **Consommation d'énergie** : Circuits embarqués ou fonctionnant sur batterie (CMOS).
3. **Compatibilité** : Compatibilité avec d'autres circuits existants (TTL, HCT CMOS).
4. **Coût** : Applications grand public ou à faible budget.

3. Analyse et synthèse de circuits logiques combinatoires

L'analyse et la synthèse des circuits logiques combinatoires sont deux aspects fondamentaux dans la conception des circuits numériques.

L'analyse d'un circuit logique combinatoire

La synthèse d'un circuit logique combinatoire a pour but la réalisation d'une fonction logique qui remplit un cahier des charges et qui satisfait également à d'autres critères tels que le coût et l'encombrement minimum par exemple. Le nombre de circuits à produire, le matériel à disposition, le délai de réalisation, etc. sont d'autres paramètres dont il faut tenir compte lors de la synthèse. De façon générale, la simplification d'un circuit est toujours utile. Concrètement, il s'agit de déterminer le logigramme associé à une fonction logique connaissant la définition de cette dernière.

¹. http://www.gecif.net/articles/genie_electrique/ressources/les_familles_logiques.pdf

Voici les étapes à suivre pour réaliser la synthèse d'un circuit logique combinatoire :

1. **Identifier les entrées et les sorties** : repérer les variables d'entrée et les variables de sortie du circuit.
2. **Étudier la structure du circuit** :
 - Identifier les portes logiques utilisées (ET, OU, NON, XOR, NAND, etc.).
 - Noter la connexion entre les portes.
3. **Écrire l'équation logique** : représenter le circuit sous forme d'une équation booléenne en analysant la manière dont les entrées sont combinées pour produire les sorties.
4. **Créer la table de vérité** :
 - Lister toutes les combinaisons possibles des entrées (2^n combinaisons pour n entrées).
 - Calculer les sorties pour chaque combinaison en appliquant les équations logiques.

La synthèse des circuits logiques combinatoires

La synthèse consiste à concevoir un circuit logique combinatoire à partir de spécifications, telles qu'une équation logique ou une table de vérité.

Étapes de la synthèse :

1. **Définir les entrées et sorties** : identifier les variables d'entrée nécessaires et la ou les variables de sortie à concevoir.
2. **Créer la table de vérité** :
 - Énumérer toutes les combinaisons possibles des entrées.
 - Indiquer les valeurs de sortie pour chaque combinaison selon les spécifications.
3. **Déduire l'équation logique** :
 - Utiliser les **formes normales** :
 - **Forme normale disjonctive (SOP)** : Somme de produits.
 - **Forme normale conjonctive (POS)** : Produit de sommes.
 - Exemple (SOP) : $F = A \wedge B \vee \neg A \wedge C = A \wedge B \vee \neg A \wedge C$.
4. **Minimisation de la fonction logique** :
 - Simplifier l'équation logique pour réduire le nombre de portes logiques nécessaires.
 - Techniques utilisées :
 - **Algèbre booléenne.**
 - **Tableau de Karnaugh (K-map)** : Une méthode visuelle pour minimiser les fonctions logiques.
 - **Méthode de Quine-McCluskey** : Une méthode systématique pour minimiser les fonctions booléennes.
5. **Dessiner le schéma logique** : représenter le circuit avec des portes logiques en fonction de l'équation simplifiée.
6. **Vérifier** la correspondance entre la table de vérité spécifiée et celle obtenue à partir du circuit conçu.

Additionneurs



Un additionneur est un circuit qui réalise l'addition de deux nombres binaires.

Nous allons dans cette section voir comment construire un circuit pour l'addition de 2 nombres en binaire.

Ce circuit étant assez complexe, nous allons le réaliser en plusieurs étapes :

- Le demi-additionneur fera une simple addition de deux bits.
- L'additionneur complet devra ajouter à cette addition celle d'un report précédent.
- Enfin nous assemblerons n additionneurs pour faire l'addition de nombres de n bits.

1. Demi-additionneur



Le demi-additionneur effectue la somme de deux bits. S est la somme et R le report (carry). Le demi-additionneur ne tient pas compte d'une retenue antérieure.

Table de vérité d'un demi-additionneur

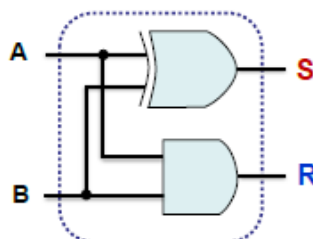
A	B	R	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Équations de sortie d'un demi-additionneur

$$R = A \cdot B$$

$$S = \bar{A} \cdot B + A \cdot \bar{B} = A \oplus B$$

Logigramme d'un demi-additionneur



Il est clair en effet que la somme est le XOR (OU Exclusif) entre A et B , et la retenue ne peut intervenir que lorsque A et B valent 1 tous les deux.

Ce circuit n'est cependant pas suffisant pour réaliser la somme de nombres de plusieurs bits. Car il ne prend pas en compte une éventuelle retenue provenant du résultat de l'addition des 2 bits de rang directement inférieur.

2. Additionneur complet

Addition complète sur 1 bit

Pour tenir compte du report précédent, il faut prévoir un circuit avec 3 entrées et 2 sorties.

Un additionneur complet comporte donc 3 entrées : les deux bits à additionner a_i et b_i , et la retenue issue de l'addition de deux bits de rang inférieurs (dite entrante), r_{i-1} .

Il possède 2 sorties : la somme S et la retenue sortante R_i .

La table de vérité est alors :

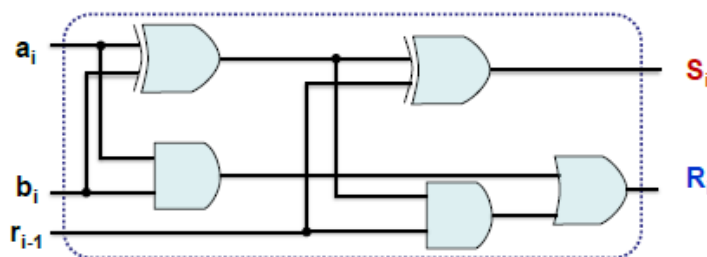
a_i	b_i	r_{i-1}	R_i	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Ses équations de sortie sont :

$$S_i = a_i \oplus b_i \oplus r_{i-1}$$

$$R_i = a_i \cdot b_i + r_{i-1}(a_i \oplus b_i)$$

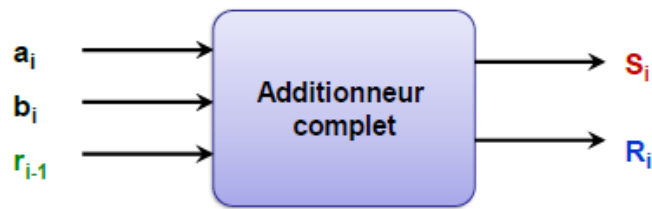
Le logigramme est :



Cette structure montre la possibilité de réaliser un additionneur complet à partir de deux demi-additionneurs et d'une **porte logique OR**.

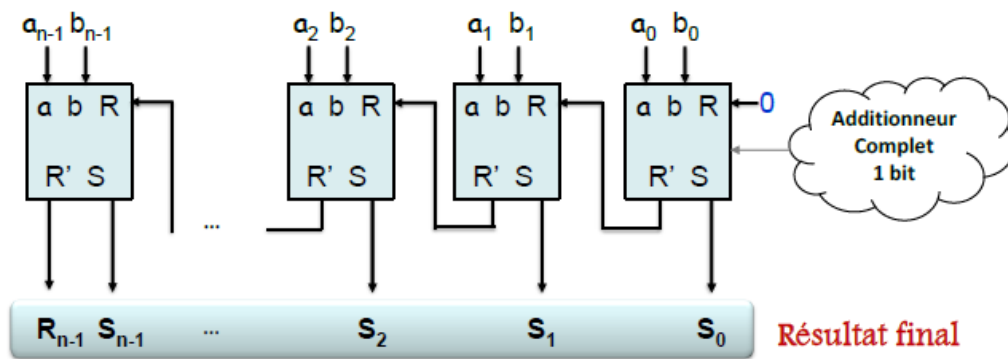
L'additionneur complet est le circuit de base pour effectuer la somme de nombres de plusieurs bits.

On peut représenter ce circuit sous la forme d'une boîte noire :



3. Addition de deux nombres binaires de n bits

L'addition de deux nombres de n bits nécessite n additionneurs. La retenue se propage des éléments binaires de poids le plus faible vers les éléments binaires de poids le plus fort. Le schéma suivant présente un exemple d'un additionneur de mots de 4 bits :



Cette architecture est intéressante d'un point de vue matériel car elle est répétitive. Par contre, le résultat obtenu dépend du nombre d'additionneurs donc de la taille des mots à additionner. La retenue R_0 est délivrée après la première addition et ainsi de suite.

Soustracteurs



Définition

Un soustracteur est un circuit qui réalise la soustraction de deux nombres binaires ($A - B$).

1. Demi-soustracteur



Définition

C'est un circuit qui fait la soustraction de deux bits b_0 et b_1 de même poids, il ne tient pas compte d'un éventuel report provenant des bits de poids inférieurs.

Table de vérité d'un demi-soustracteur

b_0	b_1	D_0	R'_0
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

D_0 représente la différence $A - B$ et R'_0 le report ou la retenue.

Équations de sortie

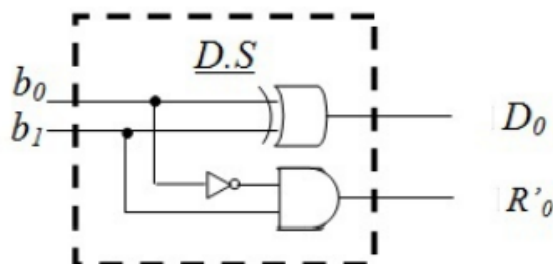
De la table de vérité sont déduites les équations de sortie :

$$D_0 = \bar{b}_0 \cdot b_1 + b_0 \cdot \bar{b}_1 = b_0 \oplus b_1$$

$$R'_0 = \bar{b}_0 \cdot b_1$$

Logigramme d'un demi-soustracteur

D'après ces équations de sortie précédentes, un demi-soustracteur est alors représenté par le logigramme suivant :



2. Soustracteur complet



Définition

C'est un circuit qui fait la soustraction de deux bits b_0 et b_1 de même poids plus le report de l'étage précédent R'_0 .

Table de vérité d'un soustracteur complet

b_0	b_1	R_0	D_1	R_1
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Équations de sortie d'un soustracteur complet

De la table de vérité du soustracteur complet, on déduit les équations de sortie suivantes :

$$D_1 = \bar{b}_0 \cdot \bar{b}_1 \cdot R_0 + \bar{b}_0 \cdot b_1 \cdot \bar{R}_0 + b_0 \cdot \bar{b}_1 \cdot \bar{R}_0 + b_0 \cdot b_1 \cdot R_0$$

$$D_1 = R_0 \cdot (\bar{b}_0 \cdot \bar{b}_1 + b_0 \cdot b_1) + \bar{R}_0 \cdot (\bar{b}_0 \cdot b_1 + b_0 \cdot \bar{b}_1)$$

Soit :

$$D_1 = R_0 \oplus (b_0 \oplus b_1)$$

En procédant de la même manière, on trouve :

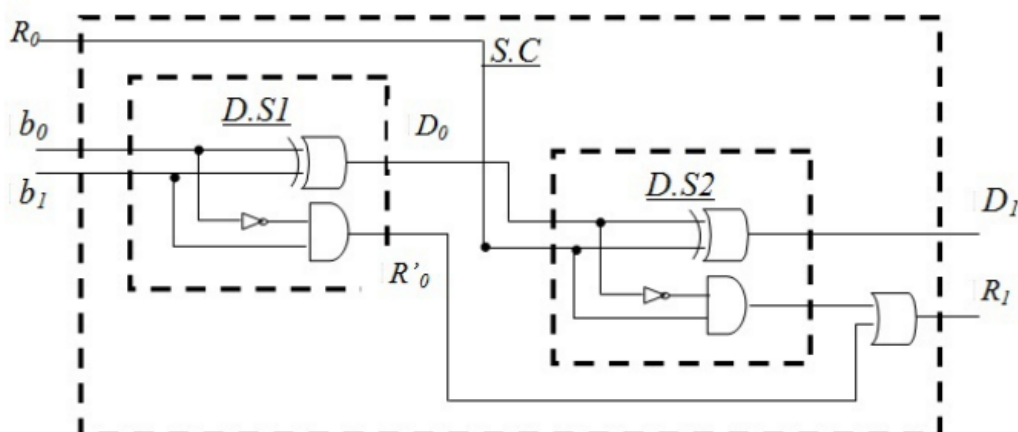
$$R_1 = \bar{b}_0 \cdot b_1 + R_0 \cdot (b_0 \oplus b_1)$$

Logigramme d'un soustracteur complet

À partir d'un demi-soustracteur, on peut obtenir un soustracteur complet.

On a ainsi : $R_1 = R'_0 + R_0 \cdot \bar{D}_0$.

Le logigramme relatif à cette équation se présente comme suit.



Multiplexeurs



Définition

Un multiplexeur est un circuit qui a pour rôle de faire circuler sur une seule voie les informations provenant de plusieurs sources.

Un multiplexeur de façon générale, possède n **entrées** de commandes (d'adresses ou de sélection) qui permettent de sélectionner l'une des 2^n **entrées de données possibles** et de l'envoyer vers l'unique sortie.

La sortie d'un multiplexeur est égale à une des entrées, le multiplexeur réalise donc un aiguillage de l'une de ses entrées vers une sortie unique selon les bits d'adresse.

Par exemple si on a 4 ($= 2^2$) entrées d'informations, nous allons avoir 2 **entrées d'adresse**. Pour la combinaison (00) des entrées d'adresse, c'est l'entrée D_0 (la première entrée) qui sera aiguillée vers la sortie. Et pour la combinaison (01), c'est l'entrée D_1 qui sera aiguillée vers la sortie et ainsi de suite.

Le schéma du multiplexeur est le suivant.

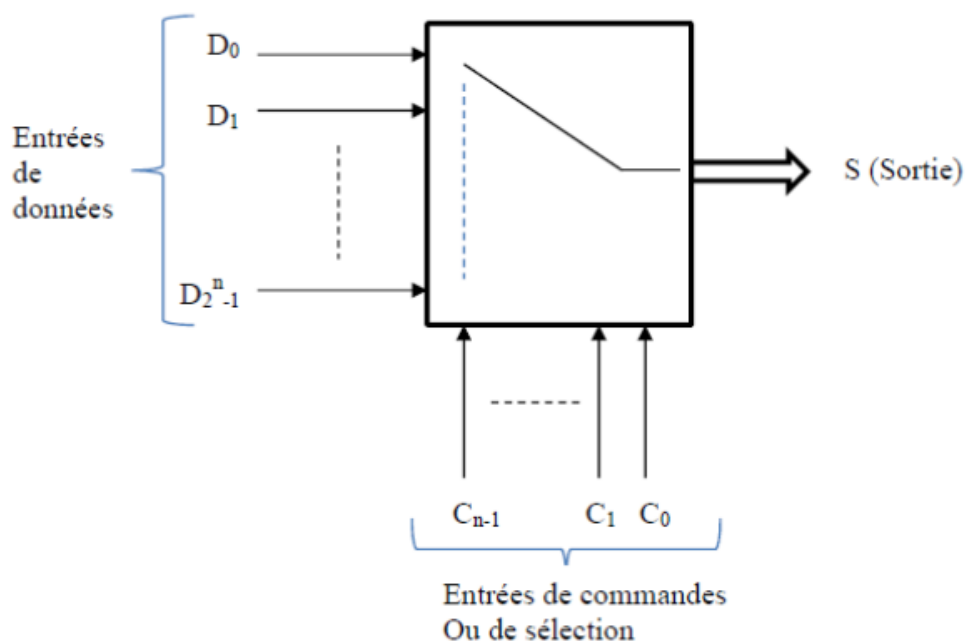


Schéma général d'un multiplexeur



Remarque

Les entrées de données peuvent avoir une taille $m \geq 1$.

La sortie S du multiplexeur doit avoir la même taille que les données.

1. La fonction du multiplexage

À partir de cette présentation fonctionnelle du multiplexeur, on va maintenant déterminer la fonction logique réalisée par ce type de circuit.

Considérons la table de vérité d'un multiplexeur à deux entrées de commande :

C_1	C_0	S
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

Cette table nous permet de déterminer l'équation logique de S :

$$S = m_0 D_0 + m_1 D_1 + m_2 D_2 + m_3 D_3$$

où m_i (minterme numéro i) représente le produit qui vaut 1 pour la combinaison d'entrée ($S_1; S_0$) tel que les valeurs associées à D_1 et D_0 forment un nombre binaire unique égal à i . C'est-à-dire :

$$S = \overline{C_1} \overline{C_0} D_0 + \overline{C_1} C_0 D_1 + C_1 \overline{C_0} D_2 + C_1 C_0 D_3$$

En généralisant ce raisonnement à **2^n entrées**, la sortie d'un multiplexeur à **n entrées d'adresses** s'exprime en fonction des données d'entrées de sélection **D_i** et des mintermes **m_i** comme suit :

$$S = \sum_{i=0}^{2^n-1} D_i m_i$$

2. Multiplexeur 4 vers 1

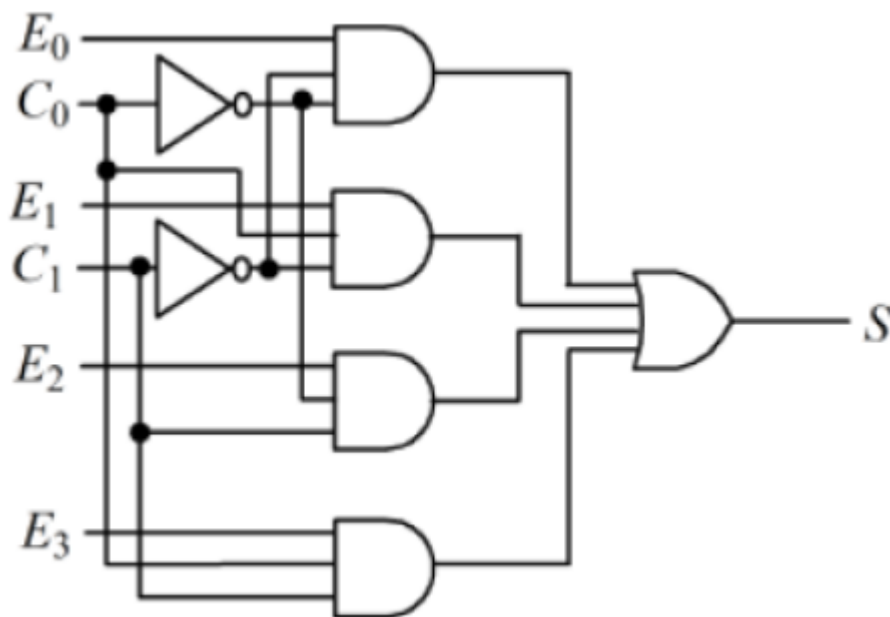
Ce multiplexeur contient 4 entrées d'informations ($4 = 2^2$), représentées par E_0, E_1, E_2 et E_3 contient donc **2 entrées de sélection** (C_1 et C_0) et une seule sortie (S). La table de vérité de ce multiplexeur est représentée ci-dessous.

Adresses		Sortie
C_1	C_0	S
0	0	E_0
0	1	E_1
1	0	E_2
1	1	E_3

L'équation logique de S est la suivante :

$$S = \overline{C_1} \overline{C_0} E_0 + \overline{C_1} C_0 E_1 + C_1 \overline{C_0} E_2 + C_1 C_0 E_3$$

Son logigramme est donné par le schéma suivant :



Circuit logique d'un multiplexeur 4 vers 1

3. Multiplexeur à 8 entrées (Mux 74151)

Ce multiplexeur dispose d'une entrée **VALIDATION**, et il fournit la sortie normale et une sortie complémentée. Quand $\bar{E} = 0$, les entrées de sélection choisissent une entrée de données (I_0 à I_7) dont les valeurs se retrouvent sur la sortie Z .

Schéma fonctionnel

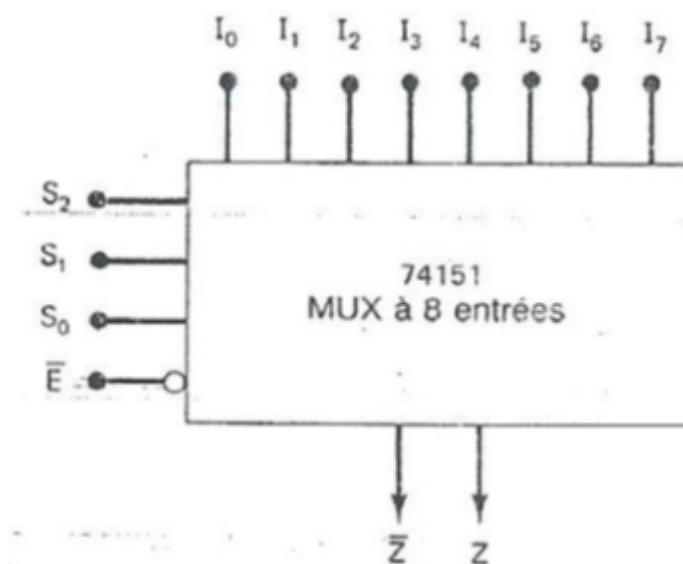


Schéma fonctionnel du Mux 74151

Table de vérité

Entrées (Adresses)				Sorties	
$\bar{E}$	S_2	S_1	S_0	$\bar{Z}$	Z
1	X	X	X	1	0
0	0	0	0	$\bar{I}_0$	I_0
0	0	0	1	$\bar{I}_1$	I_1
0	0	1	0	$\bar{I}_2$	I_2
0	0	1	1	$\bar{I}_3$	I_3
0	1	0	0	$\bar{I}_4$	I_4
0	1	0	1	$\bar{I}_5$	I_5
0	1	1	0	$\bar{I}_6$	I_6
0	1	1	1	$\bar{I}_7$	I_7

Logigramme du Mux 74151

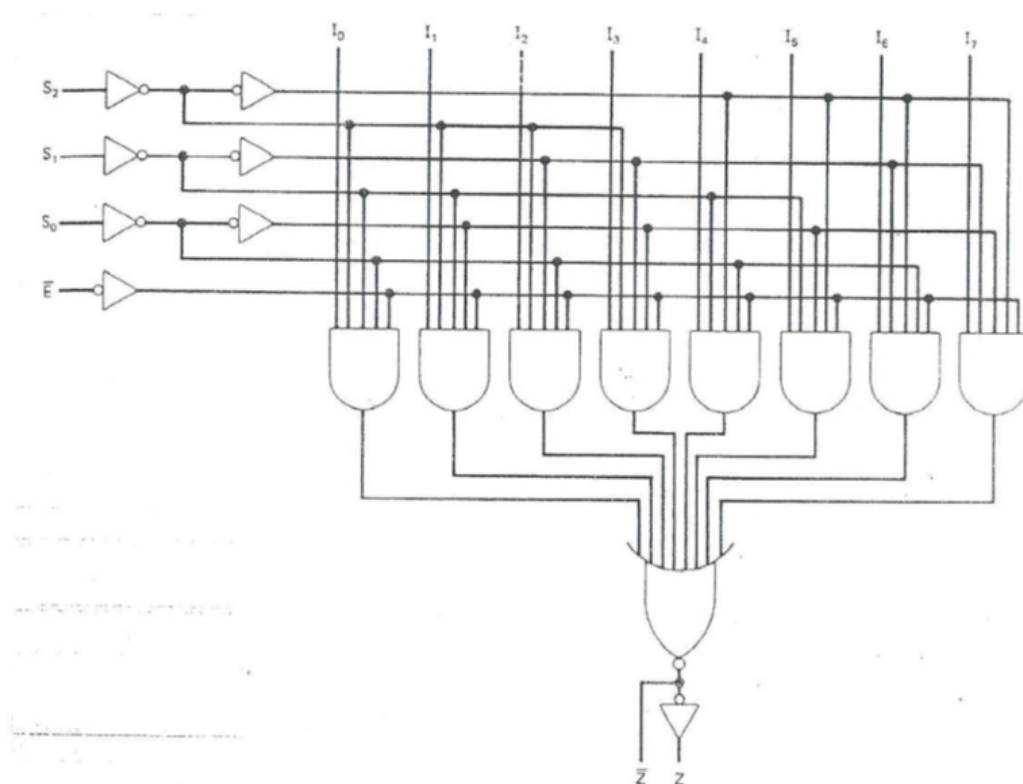


Schéma logique du Mux 74151

 Remarque

On peut se procurer dans le commerce des multiplexeurs à deux, quatre, huit et seize entrées fabriquées selon la technologie TTL ou CMOS. Ces circuits intégrés de base peuvent être combinés afin de multiplexer un nombre d'entrées encore plus grand.

Démultiplexeurs



Définition

Le démultiplexeur réalise la fonction inverse du multiplexeur. Il permet de faire passer une donnée dans l'une des 2^n sorties selon les valeurs des entrées de commande ou d'adresses (n entrées d'adresses).

Le module sélection ou adressage joue le même rôle que le multiplexeur. Il permet de sélectionner la sortie qui reçoit l'information de l'entrée. Un démultiplexeur (DeMux) possède :

- une seule entrée ;
- $N = 2^n$ sorties ;
- n entrées de sélection (commande).

Schéma fonctionnel d'un démultiplexeur

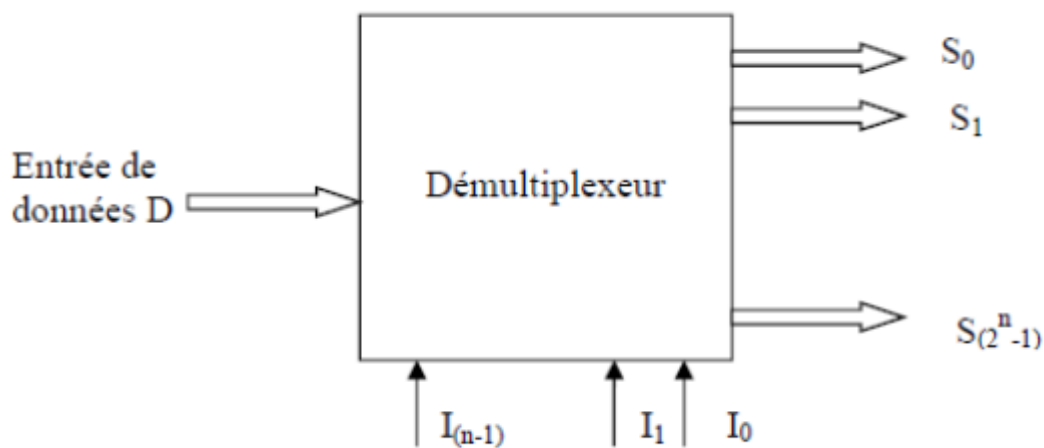


Schéma général d'un démultiplexeur

Table de vérité du démultiplexeur

Sa table de vérité est simple :

- si l'entrée D est inactive, aucune des sorties S n'est active ;
- si l'entrée D est active, la sortie active est celle dont l'indice est le numéro codé par les bits de sélection ($I_0 I_1 \dots I_n$).

1. Démultiplexeur 1 vers 4

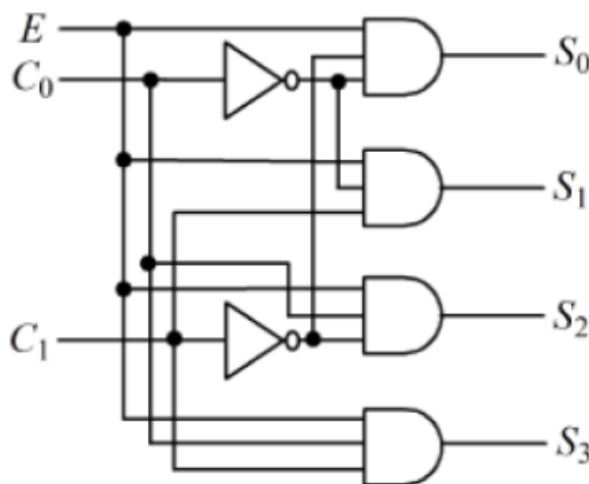
Ce circuit contient 1 entrée (E) et 4 ($= 2^2$) sorties (S_0, S_1, S_2, S_3) et 2 entrées de sélection ou d'adressage (C_0 et C_1). La table de vérité du démultiplexeur 1 vers 4 est la suivante.

C_0	C_1	S_0	S_1	S_2	S_3
0	0	E	0	0	0
0	1	0	E	0	0
1	0	0	0	E	0
1	1	0	0	0	E

Les équations logiques des sorties sont données par :

$$S_0 = \overline{C_0} \cdot \overline{C_1} \cdot E; S_1 = \overline{C_0} \cdot C_1 \cdot E; S_2 = C_0 \cdot \overline{C_1} \cdot E; S_3 = C_0 \cdot C_1 \cdot E$$

Le logigramme du démultiplexeur 1 vers 4 est le suivant :



Circuit logique d'un démultiplexeur 1 vers 4

2. Application des multiplexeurs et des démultiplexeurs

Les applications des multiplexeurs dans le domaine des techniques numériques sont nombreuses et variées. On en retrouve dans les circuits de **sélection de données**, d'**aiguillage de données**, d'**ordonnancement des opérations**, de **conversion parallèle-série**, de **génération de formes d'ondes** et de **production de fonction logique**.

- **Générateur de fonction logique**

Toute fonction logique de n variables est réalisable avec un multiplexeur de 2^n vers 1. Les entrées de sélection (**commande**) sont alors les **variables de la fonction**.

- **Conversion parallèle-série**

Considérons un mot de n bits, il peut être transmis sur un fil unique, bit après bit (transmission série) ; soit sur plusieurs fils à la fois, un fil par bit (transmission parallèle).

La **conversion parallèle-série** est effectuée à l'aide d'un multiplexeur : on envoie en entier les **n bits** du mot à transmettre, et en même temps, on fait varier les bits d'adresses en les incrémentant. En sortie, on obtient la **série** des **n bits** du mot.

La **conversion série-parallèle** est effectuée à l'aide d'un démultiplexeur : on envoie en entrée successivement les n **bits** du mot, et en même temps, on fait varier les bits d'adresses en les incrémentant. En sortie, les fils doivent être reliés à une mémoire, qui stocke l'un après l'autre les bits du mot.

Codeurs



Définition

Un codeur (ou encodeur) est un circuit logique qui traduit les valeurs d'une entrée dans un code choisi. Un codeur (ou encodeur) est un circuit qui possède 2^N voies d'entrées dont une seule est activée et N voies de sorties. Il fournit en sortie le code binaire correspondant.

Ces circuits, qui possèdent 2^N entrées et N sorties, codent en binaire sur leurs sorties le numéro de l'indice de l'entrée active. En d'autres termes, l'opération de codage consiste à donner à chaque numéro ou indice (donc à chaque caractère) un équivalent binaire, c'est-à dire un mot composé d'éléments binaires.

Le schéma fonctionnel d'un codeur est présenté comme suit :

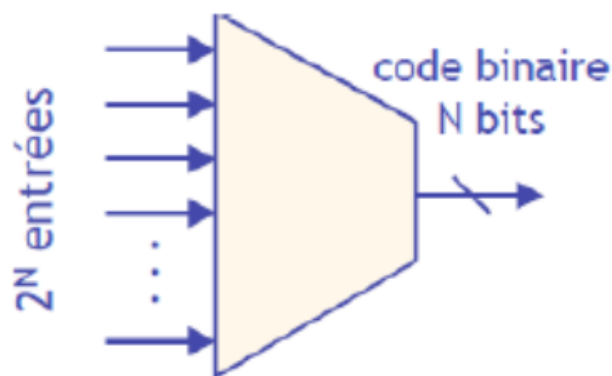


Schéma fonctionnel d'un codeur

1. Principe d'un codeurs 4 voies d'entrées et 2 bits de sortie

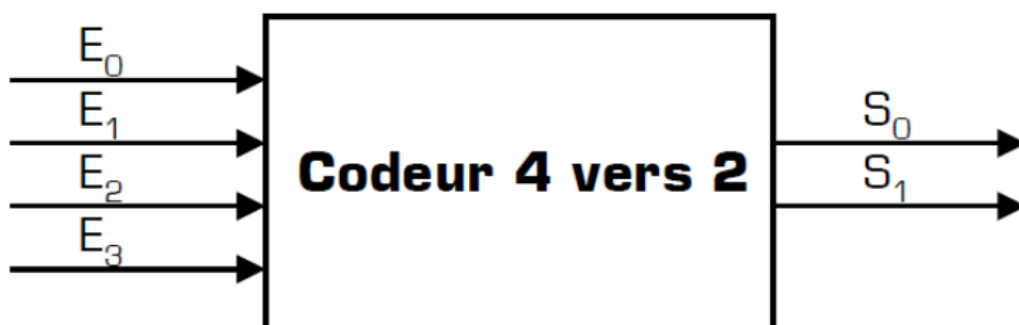


Schéma d'un codeur 4 vers 2

Ce codeur possède 4 entrées et 2 sorties. Une seule entrée doit être activée à la fois (par un état haut). On retrouve alors en sortie, en binaire, le numéro de l'entrée active entre 0 et 3.

Ici dans ce **codeur 4 vers 2**, c'est le numéro de l'indice de l'entrée active, c'est-à-dire 0, 1, 2 ou 3 qui est codé en binaire sur 2 bits. Dans ce cas, les autres entrées sont mises à 0.

Par exemple : pour le nombre décimal 2 codé en binaire vaut $(10)_2$, c'est-à-dire que $S_0 = 1$ et $S_1 = 0$.

D'où la table de vérité suivante.

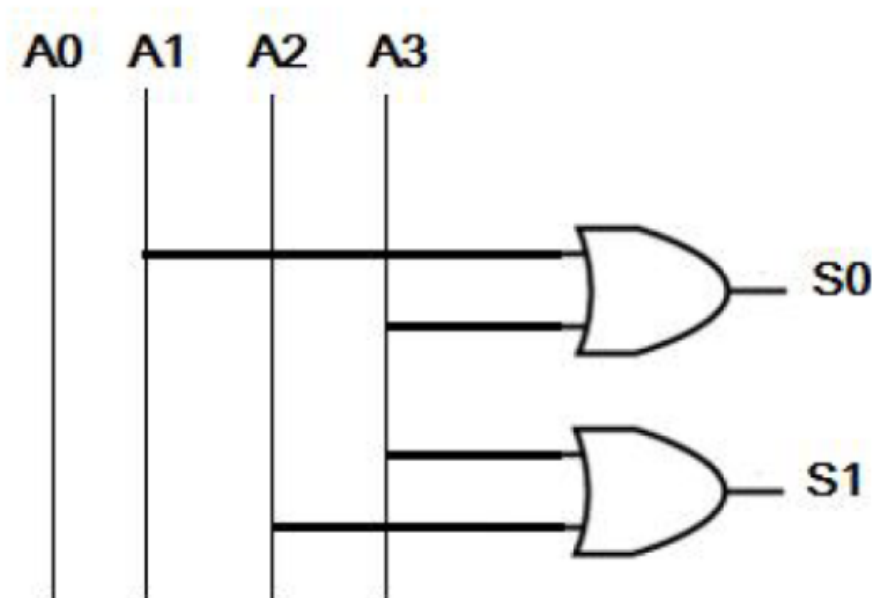
Entrées				Sortie	
A_3	A_2	A_1	A_0	S_1	S_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Les **équations logiques des sorties** sont alors :

$$S_1 = 1 \text{ si } A_2 = 1 \text{ ou } A_3 = 1 \implies S_1 = A_2 + A_3$$

$$S_0 = 1 \text{ si } A_1 = 1 \text{ ou } A_0 = 1 \implies S_0 = A_1 + A_0$$

Nous avons donc le logigramme suivant pour ce codeur 4 vers 2.



Circuit logique d'un codeur 4 vers 2

L'activation en simultanée des entrées A_1 et A_2 du codeur ci-dessous, fera passer les sorties S_1S_0 au nombre 11 qui ne correspond pas au code de l'une ou de l'autre des entrées activées. C'est plutôt le code qui représente l'activation de A_3 .

Pour résoudre ce problème on utilise un codeur de priorité qui choisit le plus grand nombre lorsque plusieurs entrées sont activées à la fois. Exemple lorsque A_1 et A_2 sont activées simultanément S_1S_0 sera égale à 10 ce qui représente l'activation de A_2 .

2. Codeur de priorité 8 vers 3



Définition

Un **codeur prioritaire** est un dispositif réalisant le codage du **numéro le plus élevé** dans le cas où **plusieurs boutons seraient actionnés**. Pour cette raison, ce codeur possède des circuits logiques en plus, de sorte que le code de sortie choisi quand deux entrées sont actives soit celui qui correspond au nombre supérieur.

Si une seule commande est envoyée sur le codeur prioritaire, celui-ci fonctionne comme un codeur classique.

Voici la table de vérité du codeur 8 vers 3.

E_7	E_6	E_5	E_4	E_3	E_2	E_1	E_0	S_2	S_1	S_0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	1	X	0	0	1
0	0	0	0	0	1	X	X	0	1	0
0	0	0	0	1	X	X	X	0	1	1
0	0	0	1	X	X	X	X	1	0	0
0	0	1	X	X	X	X	X	1	0	1
0	1	X	X	X	X	X	X	1	1	0
1	X	X	X	X	X	X	X	1	1	1

Exercice :

1. Établis l'équation de sortie de codeur.
2. Déduis-en son logigramme.

3. Codage Décimal-BCD



Définition

La fonction codage décimal-BCD consiste à convertir chaque chiffre d'un nombre décimal N_{10} en binaire dans un format de 4 **bits** chacun. Le résultat obtenu est N_{BCD} .

Code BCD



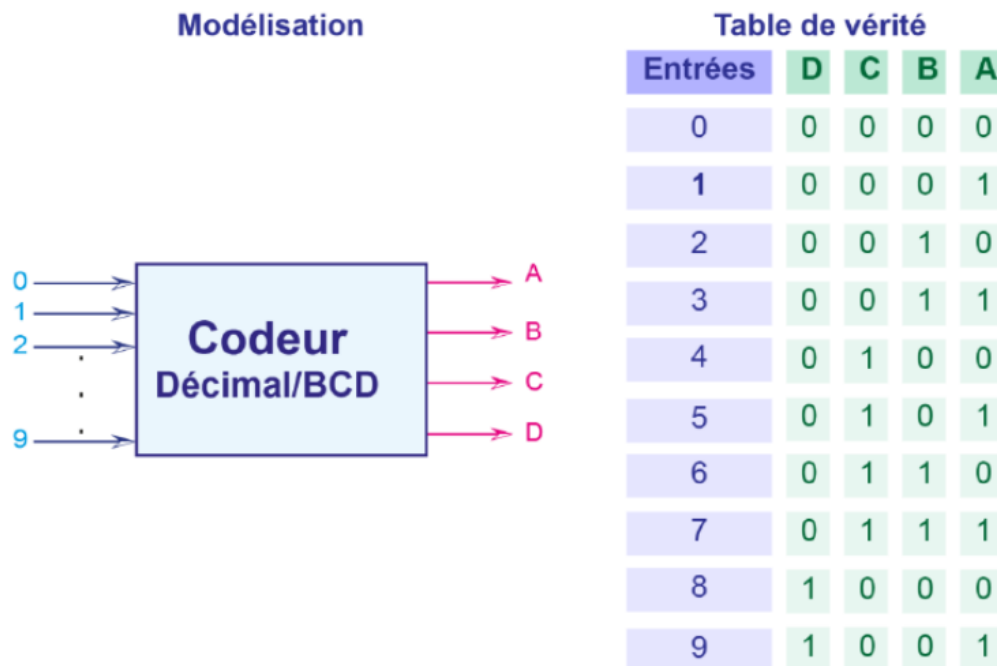
Rappel

Dans les systèmes numériques, l'entrée des données ne s'effectuent pas toujours en pur code binaire. Parfois les nombres décimaux sont codés en BCD (Binary Coded Decimal pour Décimal Codé en Binaire en français) chaque chiffre du nombre étant remplacé par son équivalent en binaire (sur 4 bits).

La codification BCD d'un chiffre décimal n'est rien d'autre qu'une substitution de ce chiffre par un groupe de quatre chiffres binaires. Ces derniers sont plus que suffisants pour coder les chiffres décimaux de «0» jusqu'à «9» donc, six (6) des seize (16) codes possibles ne seront pas utilisés dans la représentation BCD.

Modélisation et table de vérité

La fonction codage est matérialisée par un montage électronique qui reçoit en entrée 10 variables et fourni en sortie le résultat dans un format de 4 **bits** (D, C, B, A) où D est le bit de poids le plus fort et A est le bit de poids le plus faible. La figure suivante représente la modélisation et la table de vérité.



Exemple : le codeur intégré de priorité Décimal-BCD 74147

Le **circuit intégré 74LS147** (voir figure ci-dessous) est un **codeur de priorité Décimal-BCD**. Il dispose de 9 **entrées** et **quatre sorties** produisant 10 **codes BCD**. Ses entrées et ses sorties sont complémentées. Il est dit de priorité car en cas où plusieurs entrées sont simultanément actives, c'est l'entrée de plus fort poids qui l'emporte.

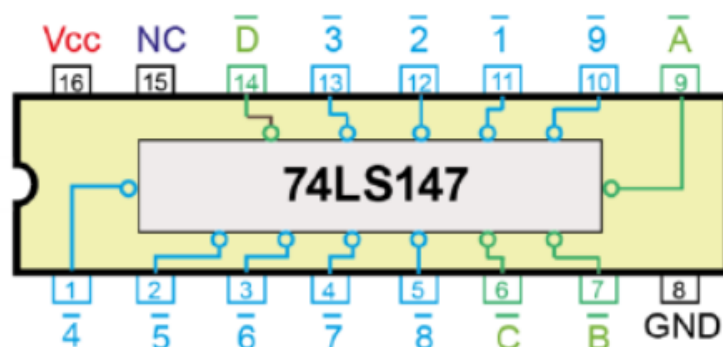


Schéma du codeur intégré de priorité Décimal-BCD 74147

La figure ci-dessous montre le circuit logique du codeur de priorité décimal-BCD 74147 ainsi que sa table de vérité. Celui-ci a 9 entrées vraies au niveau **BAS** représentant les 9 chiffres décimaux 1 à 9 et il produit à la sortie le code BCD inversé.

L'analyse de la table de vérité montre qu'une entrée est au **niveau BAS**, si toutes les entrées de rang supérieur sont au **niveau HAUT**, donne l'inverse du code BCD correspondant à cette entrée.

Les sorties de 74147 sont normalement à 1 quand aucune des entrées n'est à son **niveau vrai (bas)**. Ceci correspond à la condition d'entrée du chiffre décimal 0 (ou cela correspond au code inversé du chiffre 0). Il n'y a en réalité aucune entrée, puisque le codeur suppose que l'état d'entrée du chiffre décimal 0 est celui créé quand toutes les autres entrées sont à 1.

Pour obtenir le code BCD à partir des sorties de 74147, il faut **ajouter un inverseur à chacune des sorties**.

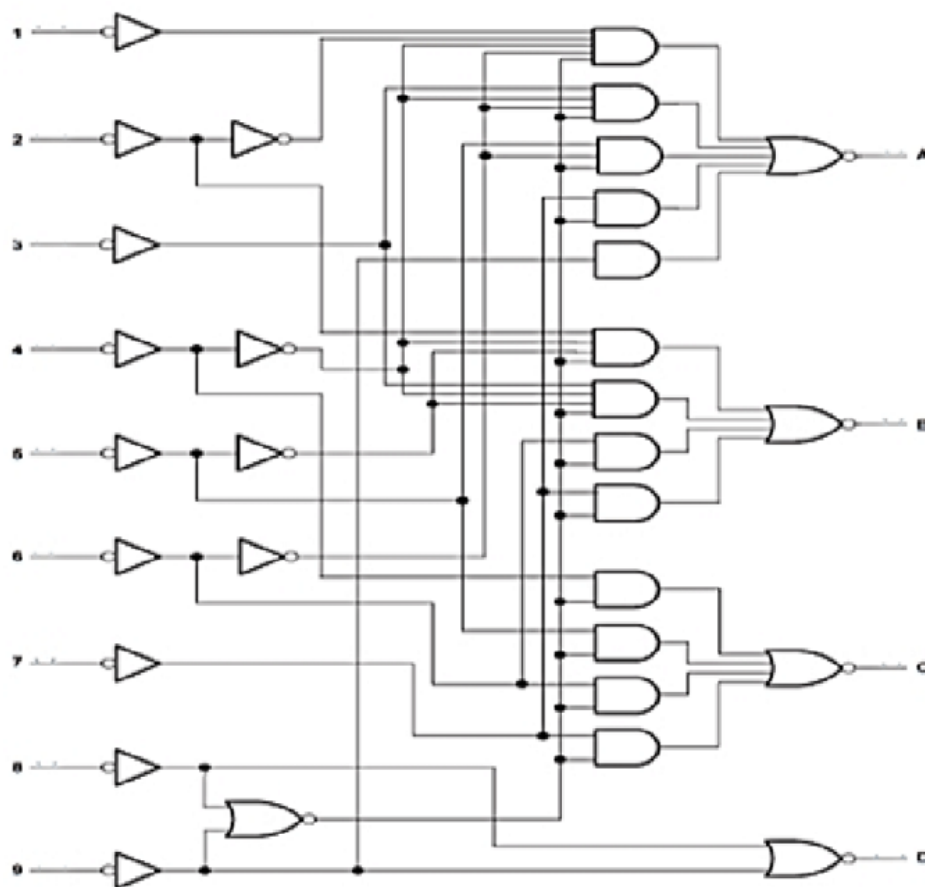


Schéma interne du circuit intégré d'un codeur Décimal-BCD

Sa table de vérité se présente comme suit.

Entrées									Sortie			
$\bar{E}_1$	$\bar{E}_2$	$\bar{E}_3$	$\bar{E}_4$	$\bar{E}_5$	$\bar{E}_6$	$\bar{E}_7$	$\bar{E}_8$	$\bar{E}_9$	$\bar{D}$	$\bar{C}$	$\bar{B}$	$\bar{A}$
1	1	1	1	1	1	1	1	1	1	1	1	1
X	X	X	X	X	X	X	X	0	0	1	1	0
X	X	X	X	X	X	X	0	1	0	1	1	1
X	X	X	X	X	X	0	1	1	1	0	0	0
X	X	X	X	X	0	1	1	1	1	0	0	1
X	X	X	X	0	1	1	1	1	1	0	1	0
X	X	X	0	1	1	1	1	1	1	0	1	1
X	X	0	1	1	1	1	1	1	1	1	0	0
X	0	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	0

Décodeurs

VII



Définition

Un décodeur est un circuit logique qui établit la correspondance entre un code d'entrée binaire de n bits et m lignes de sortie ($m \leq 2^n$). Pour chacune des combinaisons possibles des entrées une seule ligne de sortie est validée : celle dont l'indice correspond au nombre binaire appliqué sur les entrées.

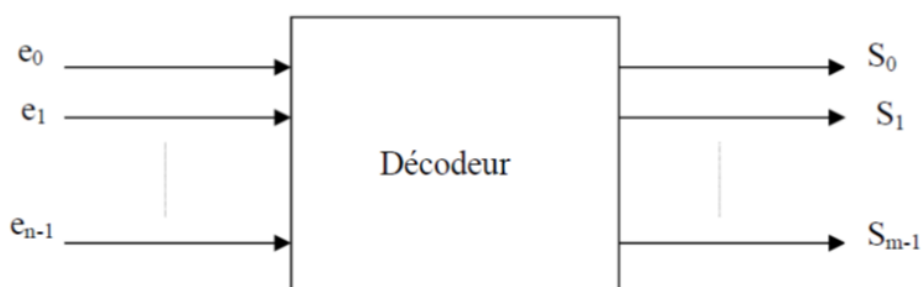


Schéma général d'un décodeur

Quand le code prend la valeur i , la sortie S_i est activée. Pour une combinaison binaire de n entrées, une seule ligne sera mise à 1.



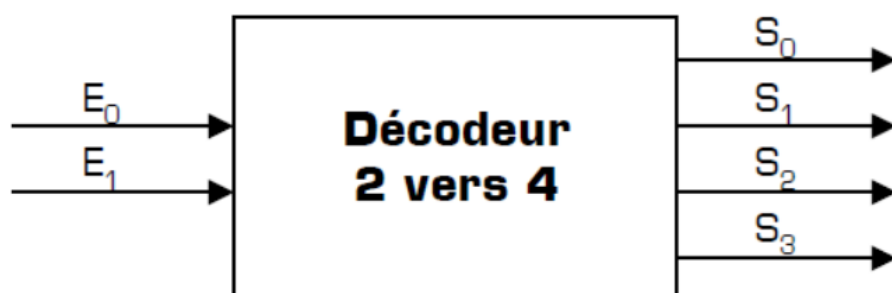
Remarque

Certains décodeurs n'utilisent pas toute la gamme de 2^n codes d'entrée possible mais seulement un sous-ensemble de ceux-ci. Ils sont alors souvent conçus de façon à ce que les codes inutilisés n'activent aucune sortie lorsqu'ils se présentent à l'entrée du décodeur (décodeur BCD-Décimal possède 4 **bits d'entrées** et 10 **sorties**).

1. Décodeur élémentaire 2 vers 4 ou décodeur 2 parmi 4

Ce décodeur possède **2 entrées** (a et b) et 4 **sorties** où chacune correspond à une des quatre combinaisons de ces variables d'entrées.

Une seule sortie est activée à la fois (par un état haut) : celle dont l'indice (entre 0 et 3) correspond au nombre (sur 2 bits) appliqué en binaire sur les entrées.



Symbole du décodeur 2 vers 4

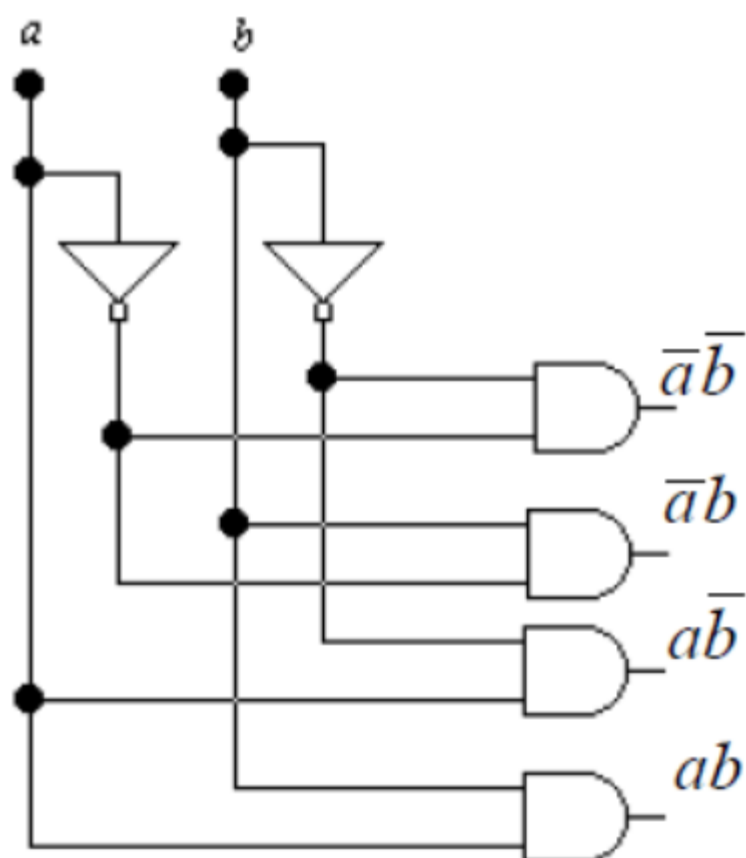
La **table de vérité** du décodeur 2 vers 4 s'établit comme suit.

a	b	S₀	S₁	S₂	S₃
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Les **équations logiques des sorties** sont :

$$S_0 = \bar{a} \cdot \bar{b}; S_1 = \bar{a} \cdot b; S_2 = a \cdot \bar{b}; S_3 = a \cdot b$$

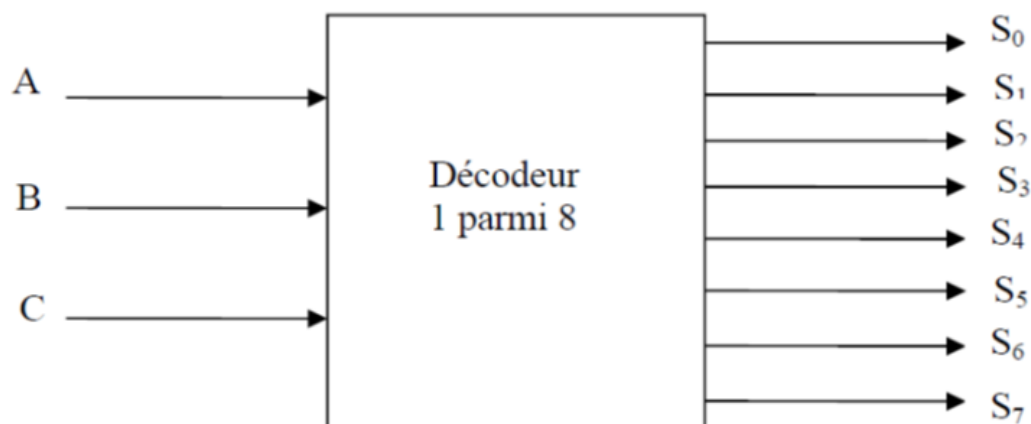
Le logigramme du décodeur 2 vers 4 est alors :



Circuit logique d'un décodeur 2 vers 4

2. Décodeur 1 parmi 8

C'est un circuit combinatoire à trois entrées et 2^3 sorties. C'est un décodeur 1 parmi 8 : 1 sortie parmi les 8 est activée à la fois.



Symbole du décodeur 1 parmi 8

La sortie activée est celle qui porte le rang de la valeur des entrées (A est la valeur de plus fort poids).

Table de vérité

A	B	C	S ₀	S ₁	S ₂	S ₃	S ₄	S ₅	S ₆	S ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Equations logiques des sorties

$$S_0 = \overline{A} \cdot \overline{B} \cdot \overline{C}$$

$$S_1 = \overline{A} \cdot \overline{B} \cdot C$$

$$S_2 = \overline{A} \cdot B \cdot \overline{C}$$

$$S_3 = \overline{A} \cdot B \cdot C$$

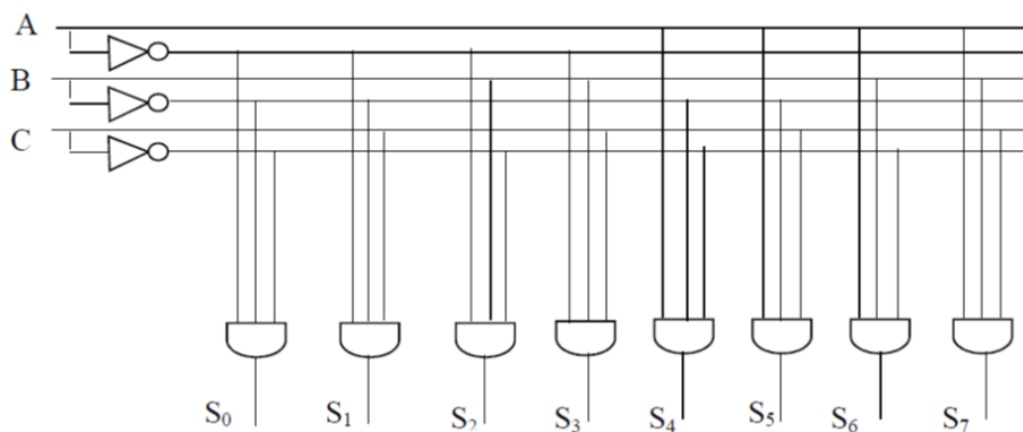
$$S_4 = A \cdot \overline{B} \cdot \overline{C}$$

$$S_5 = A \cdot \overline{B} \cdot C$$

$$S_6 = A \cdot B \cdot \overline{C}$$

$$S_7 = A \cdot B \cdot C$$

Logigramme du décodeur 1 parmi 8



Logigramme du décodeur 1 parmi 8

3. Décodage BCD-Decimal (1 parmi 10)



La fonction décodage BCD-Decimal consiste à convertir chaque quartet du nombre codé en BCD $(N)_{\text{BCD}}$ en un chiffre décimal en commençant par la droite. Le résultat obtenu est $(N)_{10}$.

Modélisation, table de vérité et équations logiques

La fonction décodage est matérialisée par un montage électronique qui reçoit en entrée 4 bits (D, C, B, A) et fournit en sortie 10 chiffres (de 0 à 9). La figure suivante représente la modélisation, la table de vérité et les équations logiques.

Modélisation

Table de vérité

D	C	B	A	Sorties
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9

Equations logiques

$$0 = \bar{A} \cdot \bar{B} \cdot \bar{C} \cdot \bar{D}$$

$$1 = \bar{A} \cdot \bar{B} \cdot \bar{C} \cdot D$$

$$2 = \bar{A} \cdot \bar{B} \cdot C \cdot \bar{D}$$

$$3 = \bar{A} \cdot \bar{B} \cdot C \cdot D$$

$$4 = \bar{A} \cdot B \cdot \bar{C} \cdot \bar{D}$$

$$5 = \bar{A} \cdot B \cdot \bar{C} \cdot D$$

$$6 = \bar{A} \cdot B \cdot C \cdot \bar{D}$$

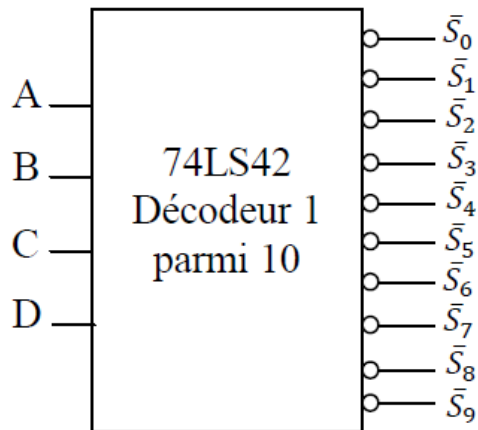
$$7 = \bar{A} \cdot B \cdot C \cdot D$$

$$8 = A \cdot \bar{B} \cdot \bar{C} \cdot \bar{D}$$

$$9 = A \cdot \bar{B} \cdot \bar{C} \cdot D$$

Exemple : le décodeur intégré BCD-decimal 74LS42A

Le circuit intégré 74LS42A (est un décodeur BCD-decimal à quatre bits d'entrée et à dix sorties $(\bar{S}_0, \bar{S}_1, \dots, \bar{S}_9)$ validées à l'état bas. Les quatre entrées A, B, C et D permettent de coder les chiffres 0 à 9.

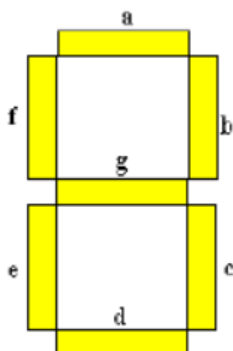


Symbole d'un décodeur BCD-décimal

Entrées				Sorties									
D	C	B	A	S ₉	S ₈	S ₇	S ₆	S ₅	S ₄	S ₃	S ₂	S ₁	S ₀
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	1	1	1	1	1	1	1	1	1	0	1
0	0	1	0	1	1	1	1	1	1	1	0	1	1
0	0	1	1	1	1	1	1	1	1	0	1	1	1
0	1	0	0	1	1	1	1	1	0	1	1	1	1
0	1	0	1	1	1	1	1	0	1	1	1	1	1
0	1	1	0	1	1	1	0	1	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1	1	1	1
1	0	0	0	1	0	1	1	1	1	1	1	1	1
1	0	0	1	0	1	1	1	1	1	1	1	1	1

4. Décodeur BCD 7 segments

Un domaine d'application considérable des décodeurs est celui de la conversion de données binaires en une forme se prêtant à un affichage numérique. Un afficheur 7 segments permet de visualiser les chiffres de 0 à 9. Cet afficheur est composé de 7 diodes électroluminescentes (D.E.L) **a**, **b**, **c**, **d**, **e**, **f** et **g** qui émettent de la lumière quand elles sont traversées par un courant et qu'ils nécessitent en fonction du type d'afficheur (anode commune ou cathode commune) une polarisation spécifique. Par exemple, pour afficher le chiffres « 3 » il faut que les segments **a**, **b**, **c**, **d** et **g** soient allumés et que les segments **e** et **f** éteints.



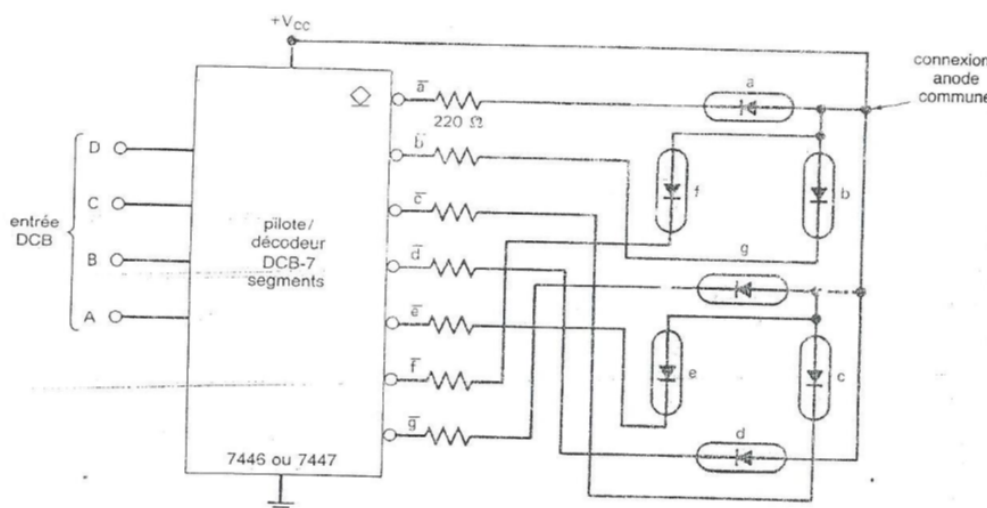
Disposition des 7 segments

Un pilote décodeur DCB-7 segments accepte en entrée les 4 **bits DCB** et rend actives les sorties qui vont permettre de faire passer un courant dans les segments qui forment le chiffre décimal

correspondant. Ici, chaque sortie, peut être au **niveau VRAI** dans plus d'une combinaison de bits d'entrée. Par exemple, **le segment e est allumé lorsque sont formés les chiffres 0, 2, 6, et 8, c'est à dire quand les codes d'entrée sont 0000, 0010, 0110 ou 1000.**

Le tableau ci-dessous donne la table de vérité détaillant le fonctionnement du décodeur BCD-7 segment permettant l'affichage des différents chiffres décimaux. Les variables d'entrées sont codées en BCD sur 4 bits A, B, C, D, la sortie fournit 7 états correspondants à chaque segment d'un afficheur nommés a, b, c, d, e, f et g. Une **sortie à 0** représente son extinction, une **sortie à 1** représente l'allumage de la LED.

Chiffre	D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	0	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	0	0	1	1



Pilote décodeur BCD 7 segments à anode commune

Le pilote/décodeur DCB-7 segments, illustré à la figure ci-dessous (TTL 7447 ou 7446), pilote un afficheur LED à 7 segments. Les anodes de ces diodes sont toutes réunies à V_{cc} (+5 V). Leurs cathodes sont connectées au travers de résistances limitatrices de courant aux sorties appropriées du pilote/décodeur. Celui-ci des sorties qui sont vraies au niveau BAS.

Pour illustrer le fonctionnement de ce circuit, soit l'entrée DCB : $D = 0$, $C = 1$, $B = 0$, $A = 1$, correspondant à la représentation DCB de 5. En réponse à cette entrée, les sorties $\bar{a}$, $\bar{f}$, $\bar{g}$, $\bar{c}$ et $\bar{d}$, et du pilote/ décodeur sont amenées au niveau BAS (raccordées à la masse), ce qui a pour effet de faire passer un courant à travers les segments LED a, f, g, c et d et d'afficher le chiffre 5. Les sorties et demeurent au niveau HAUT (ouvert) et les segment correspondantes b et e restent éteints.

Transcodeurs



Définition

Un **transcodeur** transforme une information disponible en entrée sous forme donnée (généralement un code) en la même information, mais sous une forme (généralement un autre code).

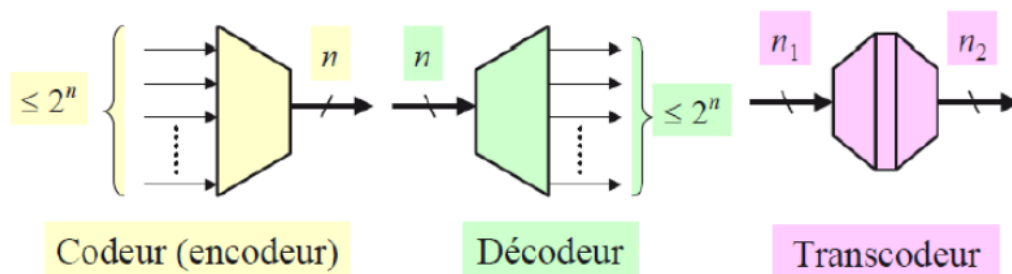


Schéma d'un transcodeur

Remarque

Il existe trois types de transcodeurs :

- un **codeur** est un **transcodeur** avec 2^n entrées et n sorties ;
- un **décodeur** est un **transcodeur** avec n entrées et 2^n sorties ;
- un **transcodeur** est un circuit de transcodage de n_1 entrées vers n_2 sorties.

1. Réalisation pratique d'un transcodeur

Comme pour la réalisation de tous les circuits en logique combinatoire, la conception d'un transcodeur passe par les 3 étapes suivantes :

- Écriture de la table de vérité complète, indiquant la valeur de toutes les sorties du circuit, en fonction de la valeur des entrées.
- Recherche et simplification des équations logiques de chacune des sorties. La simplification peut se faire soit en utilisant les tableaux de Karnaugh, soit par simplification algébrique en utilisant les propriétés de l'algèbre de Boole.
- Schéma du logigramme en utilisant les circuits (portes logiques) dont on dispose.

Parmi les transcodeurs que l'on trouve en circuits intégrés, on peut citer :

- les transcodeurs decimal-BCD (circuit 74147)
- les transcodeurs BCD-decimal (circuits 7442, 7445, et 4028)
- les transcodeurs XS3-decimal (circuit 7443)
- les transcodeurs Gray excédant 3 (code Gray+3) / décimal (circuit 7444)

- les transcodeurs DCB afficheur 7 segments (circuits 7448, 7511, 4543, 4511)
- les transcodeurs binaire 5 bits-DCB (circuit 74185)
- les transcodeurs DCB-binaire 5 bits (circuit 74184)

Dans la désignation d'un transcodeur, le code « décimal » signifie une seule entrée (ou sortie) active à la fois parmi 10.

Les deux plus importantes applications des transcodeurs sont : la **conversion de code** et l'**affichage par segment**.

2. Exemple de transcodeurs

Transcodeur binaire Gray

Pour passer d'un code à un autre, on utilisera un convertisseur de code. À titre d'illustration nous allons étudier le transcodeur binaire Gray.

Cherchons le circuit d'un transcodeur qui permet de convertir le code binaire 2 bits par exemple en code Gray.

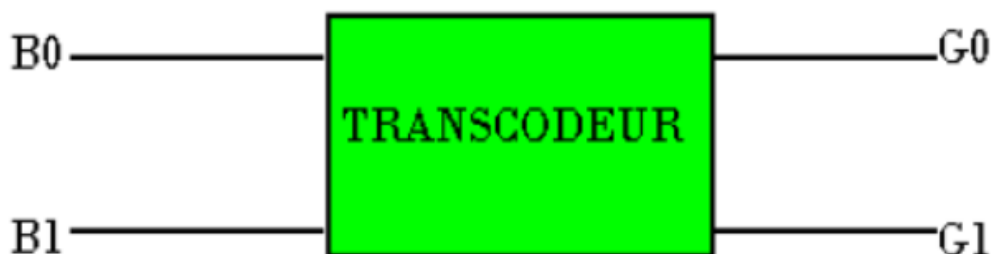


Schéma d'un transcodeur binaire

La table de vérité, les équations logiques de sortie et le logigramme sont représentés comme suit.

- **Table de vérité**

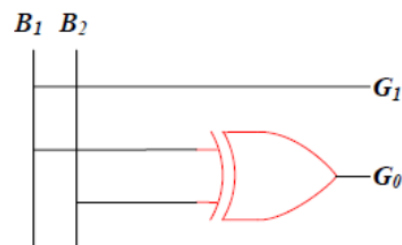
Entrées		Sorties	
B ₁	B ₀	G ₁	G ₀
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0

- **Équations de sortie**

$$G_1 = B_1 \cdot \bar{B}_0 + B_1 \cdot B_0 = B_1$$

$$G_0 = B_0 \cdot \bar{B}_1 + B_1 \cdot \bar{B}_0 = B_1 \oplus B_0$$

- **Logigramme**



Transcodeur BCD 7 segments

Un domaine d'application considérable des transcodeurs est celui de la conversion de données binaires en une forme se prêtant à un affichage numérique. Les dix chiffres 0 à 9 sont affichés au moyen d'un dispositif appelé afficheur à 7 segment lumineux qui sont des diodes électroluminescentes (D.E. L). Les variables A, B, C, D sont écrites en BCD les variables de sortie a, b, c, d, e, f, g correspondent à chacun des segments de l'afficheur.

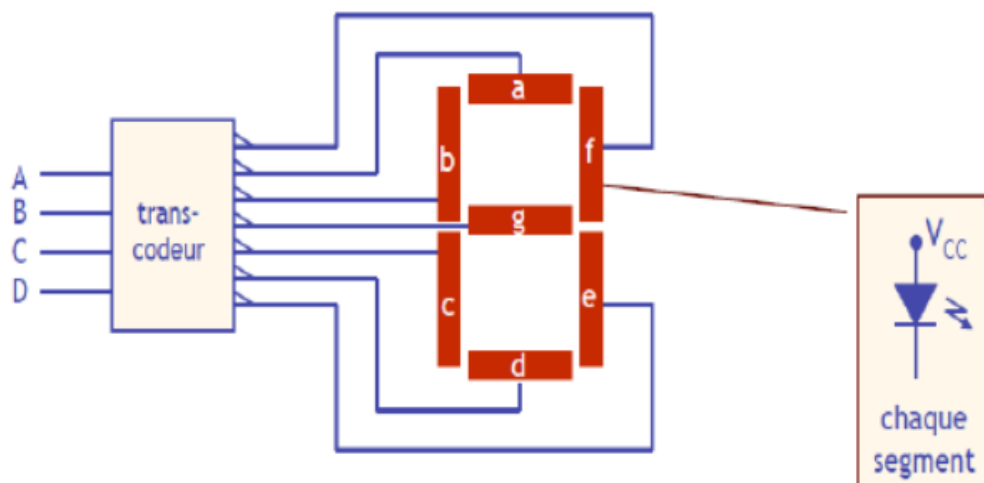


Schéma fonctionnel du transcodeur BCD 7 segments

Chiffres	ABCD	a	b	c	d	e	f	g
0	0000	1	1	1	1	1	1	0
1	0001	0	1	1	0	0	0	0
2	0010	1	1	0	1	1	0	1
3	0011	1	1	1	1	0	0	1
4	0100	0	1	1	0	0	1	1
5	0101	1	0	1	1	0	1	1
6	0110	1	0	1	1	1	1	1
7	0111	1	1	1	0	0	0	0
8	1000	1	1	1	1	1	1	1
9	1001	1	1	1	1	0	1	1

Table de vérité du transcodeur BCD 7 segments

Pour obtenir les équations logiques de ce transcodeur, il faut établir la **table de Karnaugh** relative à l'expression de chaque segment. On aura sept diagrammes. Étant donné que les 0 sont moins nombreux que les 1 dans ce diagramme de a, b, c, d, e, f, g correspondant à l'extinction des segments.

Exercice :

1. Etablis les diagrammes de Karnaugh de chaque segment.
2. Etablis l'équation logique de chaque segment à partir de son diagramme de Karnaugh.

Comparateurs



Les comparateurs logiques dits aussi circuits d'identification permettent de comparer deux nombres A et B de n bits. En général, le résultat de la comparaison est fourni sur 3 sorties :

$$A = B \implies S_{=} = 1$$

$$A > B \implies S_{>} = 1$$

$$A < B \implies S_{<} = 1$$

Deux nombres $A = a_{n-1} \cdots a_1 a_0$ et $B = b_{n-1} \cdots b_1 b_0$ sont égaux si tous les bits du même poids sont égaux.

1. Comparateur élémentaire de deux nombres de 1 bits

Il s'agit d'un circuit qui permet de comparer deux nombres de 1 bit.

Table de vérité

a	b	$S_{>}$	$S_{=}$	$S_{<}$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

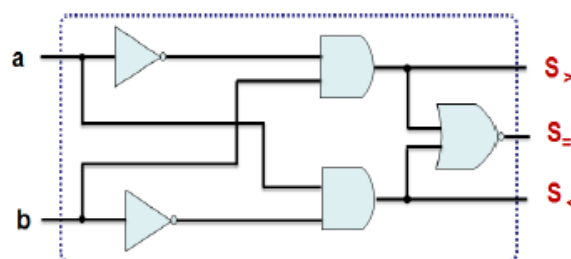
Équations de sortie

$$S_{>} = a \cdot \bar{b}$$

$$S_{<} = \bar{a} \cdot b$$

$$S_{=} = \bar{a} \cdot \bar{b} + a \cdot b = \overline{a \oplus b} = \overline{S_{>} + S_{<}}$$

Logigramme

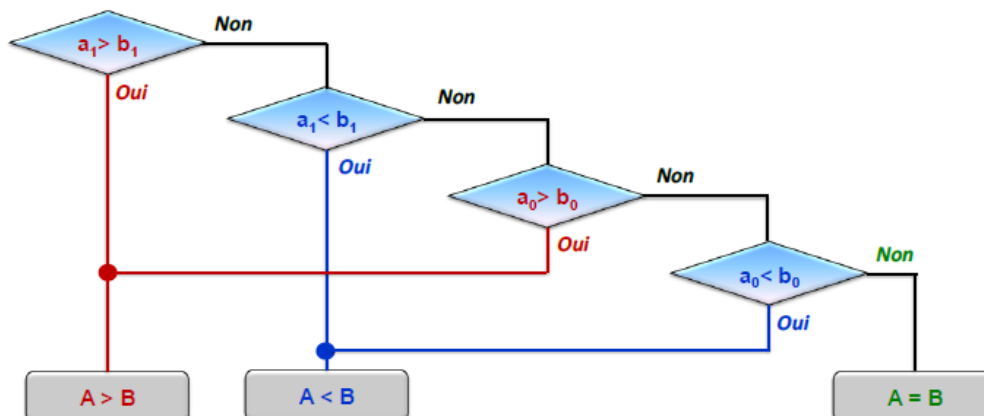


2. Comparateur de deux nombres de n bits

Principe et organigramme

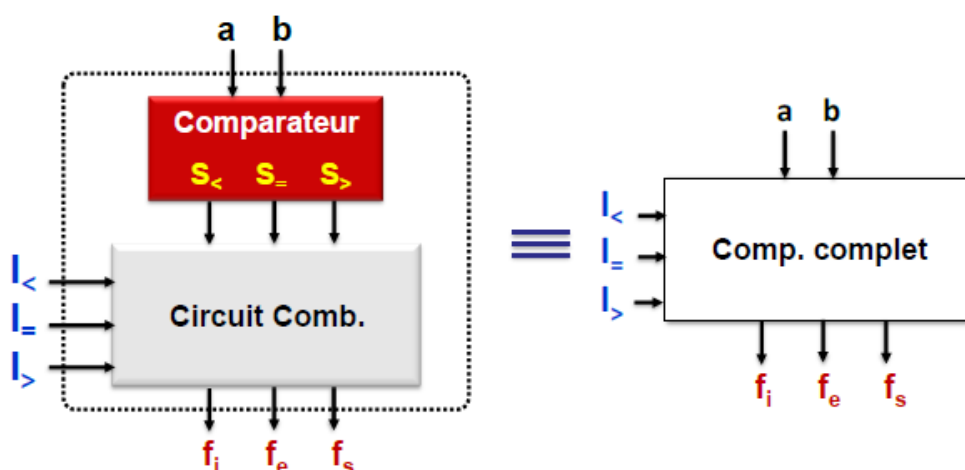


Prenant l'exemple de deux nombres A et B de deux bits : $A = a_1a_0$; $B = b_1b_0$. La démarche de comparaison est la suivante :



On commence par comparer les bits de **poinds forts** et on ne passe aux bits de poids Inférieur qu'en cas d'égalité.

La **cellule de base** de comparaison doit donc disposer d'entrées permettant la prise en compte du résultat de la comparaison des bits de **poinds inférieur**.



- $I_{<}$; $I_{=}$ et $I_{>}$: Entrées recevant le résultat de la comparaison des bits de poids inférieur.
- D'après l'organigramme, les entrées $I_{<}$; $I_{=}$ et $I_{>}$ ne sont prises en compte qu'on cas d'égalité des bits de poids supérieur ($S_{=} = 1$). Dans ce cas leur état est directement transmis vers les sorties f_i ; f_e et f_s .

Table de vérité

a	b	$S_{>}$	$S_{=}$	$S_{<}$	$I_{>}$	$I_{=}$	$I_{<}$	$I_{>}$	$I_{=}$	$I_{<}$
1	0	1	0	0	x	x	x	1	0	0
0	1	0	0	1	x	x	x	0	0	1
0	0				1	0	0	1	0	0
		0	1	0	0	1	0	0	1	0
1	1				0	0	1	0	0	1

- À partir de la table de vérité, on déduit les équations de sorties : f_i ; f_e et f_s .

$$\begin{cases} f_s = S_{>} + S_{=I_{>}} \\ f_e = S_{=I_{=}} \\ f_i = S_{<} + S_{=I_{<}} \end{cases}$$

Comparaison de deux nombres de 4 bits

? Exemple

Le comparateur 4 bits sera réalisé par la mise en cascade de 4 comparateurs de 1 bit. Le résultat de la comparaison est recueilli sur la sortie du dernier comparateur :

